

Lecture: 1010

CISC vs RISC

MUX • DEMUX • Decoders • Adders

The small library of parts every CPU is assembled from.

CSA222: Modern Computer Architecture

We Have the Machine — Now the Language

L9 organised CPU + memory. But what commands can the CPU actually understand?



The organisation

Von Neumann / Harvard — how CPU, memory & buses connect (L9).



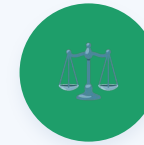
The missing piece

What set of commands does the hardware accept? That's the ISA.



The ISA

Instruction Set Architecture — the CPU's vocabulary and the HW/SW contract.



Two philosophies

CISC (rich instructions) vs RISC (simple instructions). Today's battle.

Every program you write ends up as ISA instructions. The ISA's design shapes compilers, chips, and speed.

Today's Roadmap

Two philosophies, their history, and the surprising truth about modern chips.

1

The Two Ideas

CISC: rich, variable instructions. RISC: simple, fixed ones. The core trade-off.

2

History & Compilers

VAX → MIPS → ARM → RISC-V, and what each choice costs the compiler.

3

Modern Reality

x86 secretly runs RISC-like micro-ops; RISC-V opens the ISA to everyone.

The big reveal: **the RISC-vs-CISC 'war' ended in a truce — modern CPUs borrow from both. But the ideas still shape every design.**

From Gates to a Complete Machine

A computer is more than an ALU — it needs memory, I/O, and a way to wire them together.



The question of computer organisation: how exactly do these connect — and how many buses?

The answer shapes everything: performance, cost, and how you program the machine. Two classic blueprints follow.

4. Von Neumann

One memory holds both instructions and data, reached by a single bus. The stored-program idea that started it all (1945).

The Stored-Program Concept

The radical 1945 idea: keep the program IN memory, as numbers, right alongside the data.

BEFORE vs AFTER

Before: computers were 'wired' for one task — reprogramming meant rewiring by hand.

After (von Neumann): the program is stored in the same memory as data, encoded as numbers.

*To run a different program, you just load different numbers.
Software as we know it was born.*

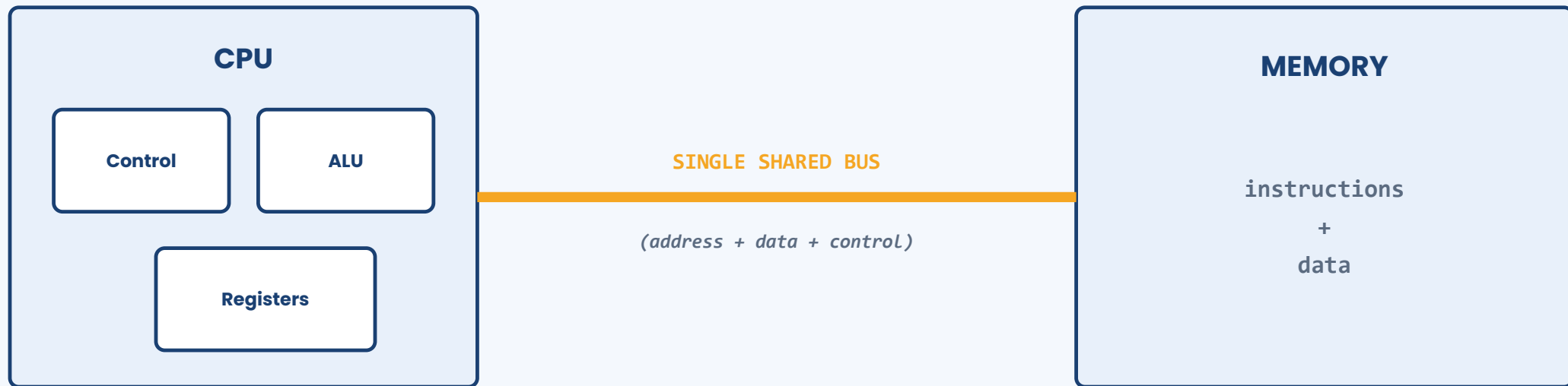
ONE MEMORY, TWO ROLES

0x00	lw r1, 0x40	instr
0x04	add r3, r1, r2	instr
0x08	sw r3, 0x44	instr
0x40	0x0000002A	data
0x44	0x00000000	data

Same memory, same address space — instructions and data live side by side.

Von Neumann Architecture

One memory, one bus — everything shares a single path between CPU and memory.



Instructions and data take turns on the ONE bus — the defining feature, and the built-in limit.

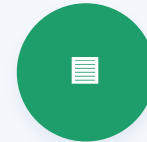
The Key Insight: Programs ARE Data

Because code lives in memory as numbers, it can be treated like any other data.



Compilers & assemblers

A program that writes another program — it outputs instructions as data into memory.



Loading & the OS

The OS loads a program by copying its instruction-bytes into memory, then jumps to them.



Just-in-time compilation

JITs (JVM, browsers) generate machine code at runtime and then execute it — data becomes code.



Self-modifying code

Programs can, in principle, rewrite their own instructions — powerful, and a security minefield.

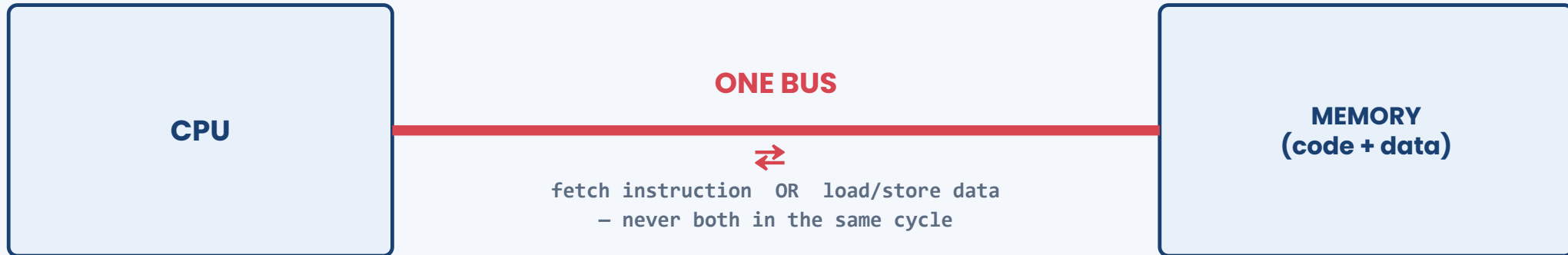
This blurring of code and data is von Neumann's superpower — and the root of whole classes of security exploits.

5. The Von Neumann Bottleneck

One bus, two jobs: instructions and data can't both cross at once. This single limit shapes modern architecture.

The Von Neumann Bottleneck

Every instruction fetch and every data access share ONE bus — so they must take turns.



The CPU can compute far faster than the single bus can feed it — so the bus becomes the limit.

This traffic jam on the memory bus is 'the Von Neumann bottleneck' — the term coined by John Backus in 1977.

Why It Limits Performance

A fast CPU stalls whenever the bus is busy — computation waits on communication.



Bus contention

Instruction fetch and data access compete for the same wires. One must wait for the other.



Idle compute units

While the ALU waits for the bus, it does nothing. Raw compute power sits unused.



Caps achievable IPC

Instructions-per-cycle is limited not by the ALU but by how fast the bus delivers work.



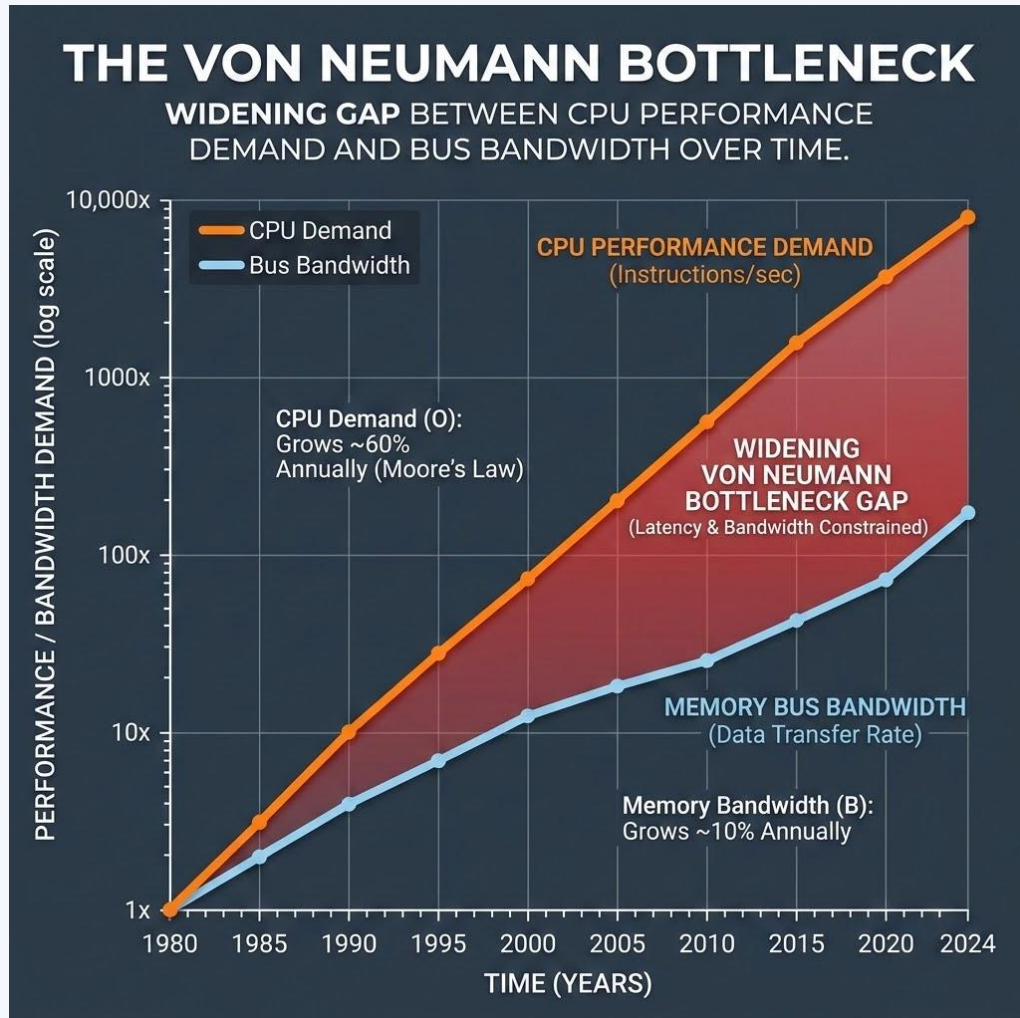
Worsens with CPU speed

Faster CPUs need data faster — but the bus doesn't scale as quickly. The gap grows.

The bottleneck is why caches, wider buses, and Harvard-style splits exist — all attempts to widen this pinch.

Why It Gets Worse Over Time

CPU speed raced ahead of memory bandwidth for decades — the bottleneck tightened.



THE RESPONSE

Architects fought back with:

- Caches — keep hot code & data on-chip (Module 5).
- Wider & faster buses — move more bits per cycle.
- Harvard-style splits — separate paths for code and data.

The rest of today — and much of this course — is about that last idea.

3. Harvard Architecture

Separate memories and buses for instructions and data — so the CPU can fetch and load at the same time.

Harvard Architecture

Two memories, two buses — instructions and data never compete for the same path.



Instruction fetch (I-bus) and data access (D-bus) happen in the SAME cycle — no contention.

Two Buses, Twice the Throughput

The whole point: parallel access. What took two cycles now takes one.

VON NEUMANN

one bus, take turns

- Cycle 1: fetch instruction.
- Cycle 2: access data.
- Two cycles for fetch + data.
- Bus is the bottleneck.

HARVARD

two buses, in parallel

- Cycle 1: fetch instruction AND access data.
- Both buses active at once.
- One cycle for fetch + data.
- Up to 2× memory throughput.

The cost: two memories and two bus systems — more wires, more pins, more silicon. Nothing is free.

One Cycle, Two Transfers

A timeline makes the advantage obvious — Harvard overlaps what Von Neumann serialises.



Same work, half the time — because the two transfers ride separate buses simultaneously.

Where Harvard Shines

When predictable, high-throughput access matters more than flexibility.



Digital Signal Processors

DSPs stream data through fixed algorithms (filters, FFTs). Parallel code+data fetch keeps the pipeline fed.



Microcontrollers

The ATmega328 (Arduino Uno) is true Harvard — flash for code, SRAM for data, separate buses.



Real-time systems

Deterministic timing matters. No bus contention means predictable, repeatable cycle counts.



Embedded & IoT

Small, fixed programs in flash + small data in RAM map naturally onto the Harvard split.

Von Neumann vs Harvard

The trade-off in one table: simplicity & flexibility vs throughput & predictability.

	VON NEUMANN	HARVARD
Memory	One (unified)	Two (separate I & D)
Buses	One shared	Two independent
Fetch + data	Sequential (2 cycles)	Simultaneous (1 cycle)
Throughput	Bottlenecked	Up to 2× higher
Cost / complexity	Lower	Higher (more wires/pins)
Flexibility	High (code = data)	Lower (separate spaces)
Typical use	General-purpose CPUs	DSPs, microcontrollers

Neither is 'better' — general CPUs pick flexibility, embedded/DSP picks throughput. Modern chips blend both...

6. Modified Harvard

What your laptop actually is: Von Neumann at main memory, Harvard inside the chip. The best of both.

The Real World: Modified Harvard

Modern CPUs are Von Neumann on the outside, Harvard on the inside.

TWO LEVELS, TWO ANSWERS

Outside the chip: one unified DRAM holds code and data — pure Von Neumann. Simple, flexible, cheap.

Inside the chip: the L1 cache is SPLIT into a separate instruction cache (L1-I) and data cache (L1-D) — Harvard-style.

So the CPU fetches code and data in parallel from L1, but everything still lives in one memory underneath.

BEST OF BOTH

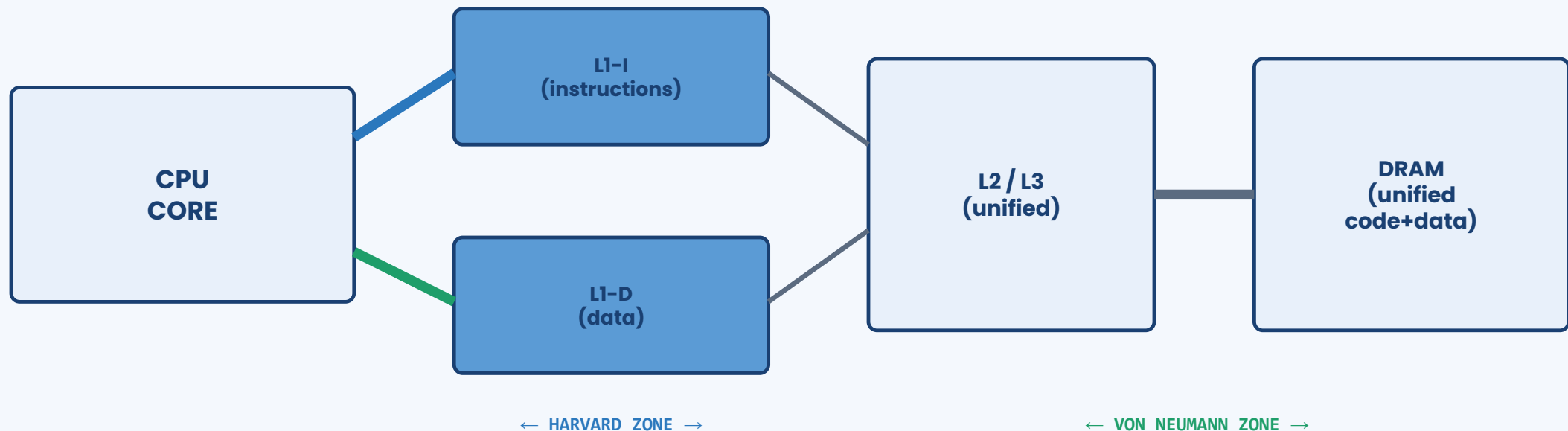
Flexibility of Von Neumann:
one address space, code = data, easy to program & load.

Speed of Harvard:
split L1 caches feed the pipeline code + data every cycle, no contention.

This hybrid is why the split L1 exists — a direct preview of Module 5.

Split L1-I and L1-D Caches

Where the two worlds meet: unified memory below, split caches above.



Split at L1 for parallel fetch; unified from L2 down for simplicity. Modified Harvard in one picture.

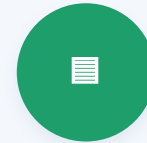
Why This Hybrid Wins

It captures each architecture's strength exactly where that strength matters most.



Parallel L1 access

Split L1-I/L1-D lets the pipeline fetch an instruction and a data word every cycle — Harvard speed where it counts.



Unified main memory

One DRAM address space keeps loading, linking, and programming simple — Von Neumann flexibility.



Cache coherence handled

Hardware keeps the split caches consistent with unified memory, so software sees one memory.



Scales to real workloads

General-purpose speed AND generality — which is why essentially every modern CPU does this.

The ISA: Hardware/Software Contract

The Instruction Set Architecture is the agreed interface — the boundary the whole industry builds around.

*everything above
compiles TO the ISA*

SOFTWARE • compilers, OS, your programs

◆ THE ISA – the instruction set ◆

HARDWARE • the CPU that implements it

*everything below
EXECUTES the ISA*

Why it's powerful: **as long as both sides honour the ISA, hardware and software teams can innovate independently. Intel can redesign a chip and your old programs still run. The ISA is the promise that makes that possible.**

1. Two Philosophies

CISC packs power into each instruction; RISC keeps every instruction simple. One trades hardware complexity for the other's compiler complexity.



1

CISC: Complex Instruction Set

Make each instruction do a lot — fewer lines of assembly, richer operations.

THE PHILOSOPHY

One instruction can do a whole task — even memory-to-memory arithmetic in a single line.

Instructions are variable-length: simple ones are short, complex ones are long.

Born when memory was scarce and expensive — fewer, denser instructions saved precious space.

One CISC instruction:

add [mem1], [mem2]

CHARACTER

- Hundreds of instructions
- Variable length (1–15 bytes on x86)
- Instructions can touch memory directly
- Complex hardware decoder
- Fewer lines of assembly
- e.g. x86, VAX, 68000

RISC: Reduced Instruction Set

Keep every instruction simple and uniform — let the compiler combine them.

THE PHILOSOPHY

Each instruction does ONE simple thing, in one cycle where possible.

All instructions are the same fixed length — easy to fetch, decode, and pipeline.

Only load and store touch memory — everything else works on registers (load/store architecture).

Same work, RISC style:

```
lw r1,m1 • lw r2,m2 • add r1,r1,r2 • sw r1,m1
```

CHARACTER

- Few, simple instructions
- Fixed length (e.g. 32 bits)
- Only load/store touch memory
- Simple, fast hardware decoder
- More instructions per task
- e.g. MIPS, ARM, RISC-V, SPARC

One Task, Two Styles (Activity 1)

Compute $C = A + B$; $D = C \times 2$ — count the instructions each way.

CISC STYLE

```
mov  ax, [A]
add  ax, [B]      ; ax = A + B
mov  [C], ax
shl  ax, 1        ; ax = C * 2
mov  [D], ax
```

~5 instructions · richer ops · direct memory operands

RISC STYLE

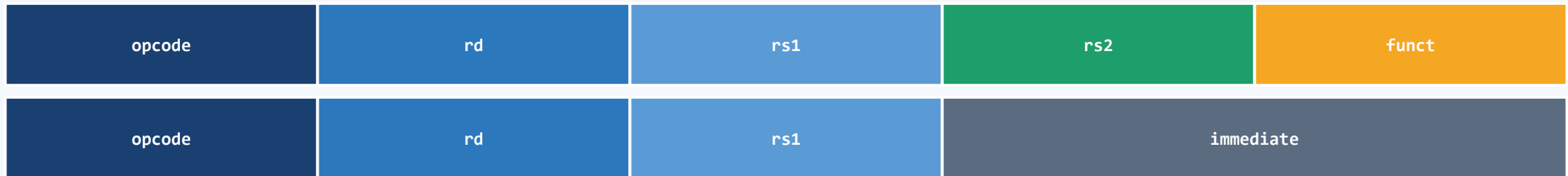
```
lw   r1, A
lw   r2, B
add  r3, r1, r2   # C = A + B
sw   r3, C
add  r4, r3, r3   # D = C * 2
sw   r4, D
```

~6 instructions · each simple & 1 cycle · load/store only

Instruction Encoding

Fixed-width vs variable-width — the choice that ripples through the whole chip.

RISC – FIXED 32-BIT (every instruction identical width)



Same length every time → trivial to find where the next instruction starts.

CISC – VARIABLE LENGTH (1 to 15 bytes on x86)



Different lengths → the CPU must decode one instruction before it knows where the next begins.

This single difference is why RISC pipelines so much more easily — predictable fetch and decode.

The Decoder Tells the Story

Where the complexity lives is the whole difference — hardware vs compiler.

CISC DECODER

complexity in **HARDWARE**

- Must handle variable-length instructions.
- Often uses microcode — a mini-program per instruction.
- Large, power-hungry, hard to pipeline.
- But: shorter programs, denser code.

RISC DECODER

complexity in the **COMPILER**

- Fixed length → decode is almost trivial.
- One instruction = one simple action.
- Small, fast, easy to pipeline & replicate.
- But: the compiler works harder to sequence more instructions.

The trade never disappears — it just moves. CISC asks more of the chip; RISC asks more of the compiler.

1990s–2000s: Two Empires

RISC conquered mobile; CISC held the desktop. Both thrived — for different reasons.

ARM — RISC wins mobile

- Simple = low power = long battery life.
- Licensable design → everyone builds ARM chips.
- Billions of phones, tablets, embedded devices.
- Now in Apple Silicon laptops too.

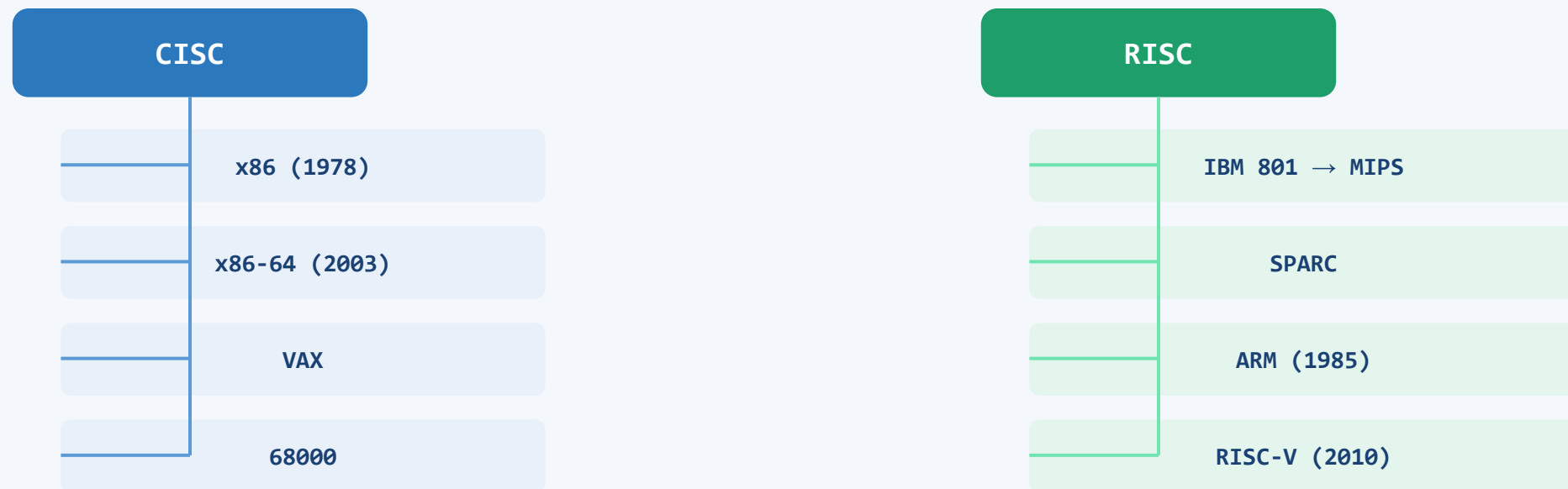
x86 — CISC holds desktop

- Backward compatibility: decades of software.
- Intel & AMD poured R&D into raw speed.
- The whole PC & server ecosystem locked in.
- Still dominant in desktops & data centres.

The lesson: 'best ISA' rarely decides the winner — ecosystems, power budgets, and compatibility do.

The ISA Family Tree

A rough map of who came from where — and which philosophy they follow.



Two lineages, one lesson: the split defined 40 years of chips — but the next slides show how they've converged.

3. Compilers & Code Density

The philosophy choice lands hardest on two things: how compact your code is, and how easily the chip can pipeline it.



3

Code Density: CISC's Edge

Fewer, richer instructions mean smaller programs — which eases pressure on the instruction cache.

WHY DENSITY MATTERS

CISC programs are often smaller in bytes — one instruction does more work.

Smaller code = fewer bytes to fetch, and less pressure on the L1 instruction cache (recall L8).

In memory-constrained or cache-sensitive settings, density is a real, measurable advantage.

PROGRAM SIZE (illustrative)

CISC



RISC



But it's not the whole story:

RISC's fixed 32-bit instructions can waste some space, yet uniformity buys huge pipelining gains. Compressed RISC ISAs (ARM Thumb, RISC-V C) close much of the density gap.

Pipelining: RISC's Edge

Uniform instructions flow through a pipeline like clockwork — the reason RISC dominates performance-per-watt.



Fixed length = easy fetch

The CPU always knows where the next instruction is — no decode needed first. Perfect for a pipeline.



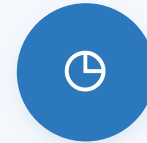
Uniform format = simple decode

Every instruction decodes the same way, in one stage, at one speed.



One cycle each = smooth flow

Simple ops rarely stall the pipeline, so stages stay full and throughput stays high.



Load/store = predictable memory

Only two instructions touch memory, so hazards are easier to detect and handle (Module 4).

Pipelining is the heart of Module 4 — and RISC's uniformity is exactly what makes it clean. Remember this slide.

The Trade-Off in One View

Every RISC strength has a matching CISC strength — it's genuinely a balance.

CISC STRENGTHS

- ✓ Denser code (smaller programs)
- ✓ Fewer instructions per task
- ✓ Less instruction-fetch bandwidth
- ✓ Rich, expressive operations

RISC STRENGTHS

- ✓ Simple, fast, cheap decoder
- ✓ Easy, efficient pipelining
- ✓ Higher clock speeds possible
- ✓ Simpler to design & verify

Because both lists are real, the industry didn't pick a winner — it learned to combine them. That's next.

4. The Modern Truth

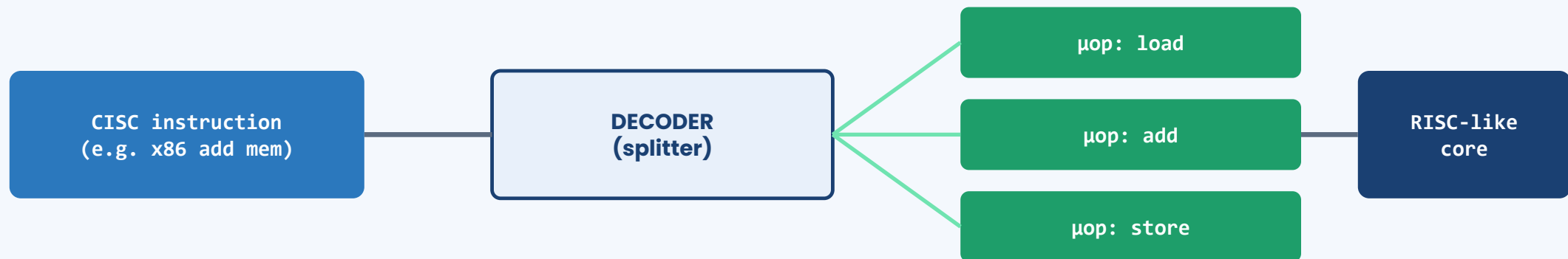
The war is over. Inside every modern x86 chip beats a RISC-like heart — the two philosophies have merged.



4

The Secret Inside x86: Micro-Ops

Modern x86 chips are CISC on the outside, RISC on the inside.



The front-end decodes each complex x86 instruction into several simple micro-ops (μops)...

...then the back-end pipelines and renames those μops exactly like a RISC machine. Best of both, in one chip.

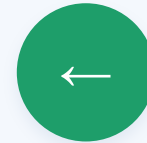
The Boundary Has Blurred

Both camps borrowed each other's best ideas until the labels stopped meaning much.



CISC went RISC inside

x86 decodes to RISC-like μ ops, then pipelines and reorders them. The CISC is just a compatibility skin.



RISC added complexity

Modern ARM/RISC-V have hundreds of instructions, SIMD, and vector extensions — not so 'reduced' anymore.



Compressed instructions

ARM Thumb and RISC-V 'C' add 16-bit encodings for density — a very CISC-like concern.



It's about implementation now

What matters today is the microarchitecture — caches, pipelines, predictors — more than the ISA label.

So Who Won?

Both — and neither. The pragmatic answer is what every modern high-performance chip does.

The modern consensus: a RISC-like execution core is the way to go for performance — so keep a clean, pipelineable core, and put whatever instruction 'skin' the market needs on the front.

x86

keeps CISC skin for compatibility, RISC core inside

ARM

RISC through and through — efficiency king

RISC-V

clean RISC, open to all, modular extensions

5. Where It's Heading

Open ISAs and domain-specific accelerators are rewriting the rules — a preview of Module 6.



5

RISC-V: The Open ISA

For the first time, a serious instruction set that anyone can use, free — no license, no royalties.



Open & royalty-free

Anyone — students, startups, giants — can design a RISC-V chip without paying anyone. Unprecedented.



Modular by design

A small base ISA plus optional extensions (multiply, float, vector, atomic). Take only what you need.



Exploding adoption

From university teaching to microcontrollers to data-centre and AI chips — growth is rapid.



Born at Berkeley

Created in 2010 by the same research lineage that gave us RISC in the first place.

We'll use RISC-V assembly in the coming lectures — its clean design makes it the perfect teaching ISA.

Beyond CPUs: Domain-Specific ISAs

The newest frontier — instruction sets built for one job, especially AI.



GPU ISAs (NVIDIA PTX)

Thousands of simple cores with their own instruction set, built for massive parallelism.



TPU / AI accelerators

Google's TPU has an ISA centred on matrix multiply — the core operation of neural networks.



Why specialise?

A fixed workload lets you strip away generality and win huge efficiency — the opposite of general-purpose.



The Module 6 preview

Multicore, GPUs, and AI accelerators are where architecture is headed. This is the door to it.

Common Mistakes to Avoid

The slips that cost marks — and reveal shaky understanding.



'RISC is always faster'

Not inherently. RISC eases pipelining, but real speed depends on the whole microarchitecture.



Forgetting load/store

The load/store rule is RISC's defining feature — only load & store touch memory.



'CISC is obsolete'

x86 still dominates desktops & servers. It just runs RISC-like μ ops inside.



'The ISA determines speed'

The implementation (pipeline, caches, predictors) matters far more than the ISA label.



'RISC = fewer instructions run'

RISC has fewer instruction TYPES, but often runs MORE instructions per task.



Mixing up length rules

RISC = fixed length. CISC = variable length. This drives decoder complexity.

Key Takeaways

Five sentences — the two philosophies and their truce.

- 01 The ISA is the hardware/software contract — the instruction set both compilers and chips agree to.
- 02 CISC uses rich, variable-length instructions (x86, VAX): dense code, complex decoder, harder to pipeline.
- 03 RISC uses simple, fixed-length, load/store instructions (MIPS, ARM, RISC-V): easy to pipeline, simpler chips.
- 04 The trade-off just moves complexity between hardware and compiler — neither philosophy escapes it.
- 05 Modern x86 decodes CISC into RISC-like micro-ops; RISC-V opens the ISA to all — the two worlds have merged.

Practice & Homework

Write assembly, compare ISAs, and debate.

01

Vector Add

Write a 3-element vector add in (a) x86-style CISC and (b) RISC-V style. Count instructions.

02

Why Pipeline?

List 3 reasons RISC is friendlier to pipelining. Tie each to a concrete instruction property.

03

Mini Debate

Two teams: 'RISC won' vs 'CISC won'. 5 minutes of arguments each — then we debrief.

EXIT TICKET

1. Name the two hallmarks of RISC (hint: instruction length + memory access).
2. What are micro-ops, and why does x86 use them?
3. Why is Apple able to use ARM (RISC) for laptops while x86 still thrives on the desktop?

Rapid-Fire Recap

Three questions — shout the answers.

Q1**RISC instructions are what length?****fixed****Q2****In RISC, only these touch memory:****load / store****Q3****x86 decodes instructions into...****micro-ops**

All three instant? You understand the ISA landscape — next we get concrete with instruction formats & assembly.

END OF L10

Simple or rich? The answer is: both.

NEXT — L11

Instruction Formats & Addressing Modes — how an instruction is actually encoded in bits, and the ways it can name its operands.

